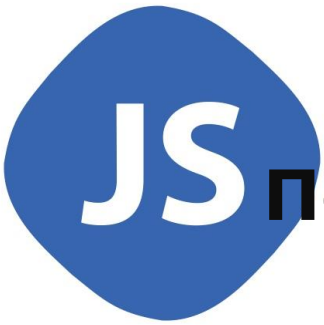


Пользовательские события

- Введение
- Конструктор Event
- Метод dispatchEvent
- Специфические конструкторы
- Пользовательские события
- Пользовательские события для коллекций
- Политика безопасности браузера
- Творческие эксперименты

Введение

Используя возможности программирования, можно не только назначать обработчики на HTML-элементы, но и выполнять **автоматическую генерацию событий из JavaScript-кода**.

Генерировать можно:

- **встроенные события** (click, mousedown и другие)
- совершенно новые, **придуманные разработчиком события** (sayhello, filldata и другие)

Пользовательские события могут быть использованы при создании графических компонентов.

> ***Примечание***. В свете предыдущих уроков необходимость пользовательских событий неочевидна. Но если вы планируете двигаться дальше и изучать асинхронный способ взаимодействия приложений, мой совет – поднимите эту тему. Пригодится.



И уж тем более, пользовательские события будут полезны, если вы планируете *пудрить мозги* пользователям контекстной рекламой всплывающих окон (об этом скоро).

Конструктор Event

Встроенные классы для событий формируют иерархию аналогично классам для DOM-элементов. Корнем этой иерархии является встроенный класс **Event**.

Событие **встроенного класса Event** можно создать так:

```
let event = new Event(type[, options]);
```

Где:

- **type** – тип события, строка, например "click" или же любой придуманный разработчиком – "myevent".
- **options** – объект с тремя необязательными свойствами:
  - **bubbles** – true/false – если true, тогда событие всплывает.
  - **cancelable** – true/false – если true, тогда можно отменить действие по умолчанию.
  - **composed** – true/false – если true, тогда событие будет всплывать наружу за пределы DOM.

По умолчанию все три свойства установлены в **false**:

```
{bubbles: false, cancelable: false, composed: false}.
```

При явном указании всех свойств, синтаксис создания события может быть таким:

```
let event = new Event(type, {
  bubbles: true,
  cancelable: true,
  composed: false
});
```



Метод dispatchEvent

После того, как объект события создан, мы должны запустить его на элементе, вызвав метод **dispatchEvent**(). Затем обработчики отреагируют на него, как будто это обычное браузерное событие. Если при создании указан флаг **bubbles**, то оно будет **всплывать**.

> ***Важно***. Метод **dispathEvent**() запускает событие, так, как если бы оно было **инициировано пользователем**.

В примере ниже события **click** и **change** инициируются JavaScript-кодом так, как будто были вызваны действиями пользователя.

example_1.

```
<!-- обработчик события click определен в HTML-атрибуте -->
<button id="btn" onclick="alert('Клик!')">Эта кнопка умеет кликать сама себя!</button><p>
Это поле умеет вызывать onchange без пользователя!<br>
<input id="inp" type="text" value="">
<script>
    // создаем событие
    var eventClick = new Event("click");
    // вызов события
    btn.dispatchEvent(eventClick);
    // определяем обработчик события change
    inp.onchange = function () {this.value='Hello my dear user!'}
    // создаем событие
    var eventChange = new Event("change");
    // вызов события
```

```
    inp.dispatchEvent(eventChange);
</script>
```

Для некоторых стандартных событий есть альтернатива методу **dispatchEvent**(). Если вам не нужны расширенные настройки, то вызов события может быть выполнен так, как показано в демонстрационном примере **example_2**.

example_2.

```
<!-- обработчик события click определен в HTML-атрибуте -->
<button id="btn" onclick="alert('Клик!')">Эта кнопка умеет кликать сама себя!</button><p>
Это поле умеет вызывать onchange без пользователя!<br>
<input id="inp" type="text" value="">
<script>
    // вызов события
    btn.onclick();
    // btn.click();
    // определяем обработчик события change
    inp.onchange = function () {this.value='Hello my dear user!'}
    // вызов события
    inp.onchange();
</script>
```

Специфические конструкторы

Для некоторых конкретных типов событий есть свои **специфические конструкторы**.

Вот только некоторые из конструкторов для различных событий пользовательского интерфейса, которые можно найти в спецификации UI Event:

- UIEvent
- FocusEvent
- MouseEvent
- WheelEvent
- KeyboardEvent
- и другие.

Стоит использовать именно эти конструкторы вместо **new Event**, если мы хотим создавать такие события.

Пример:

```
let event = new MouseEvent("click");
```

Специфический конструктор позволяет указать **стандартные свойства для данного типа события**. Например, для события мыши **MouseEvent** стандартными являются свойства clientX/clientY элемента.

example_3.

```
<a id="link" href="https://vk.com/pechora_pro">Переход на vk.com/pechora_pro</a>
<script>
    // получаем елемент с id=link
    const link = document.getElementById("link");
    // определяем обработчик клика по ссылке
    link.onclick = function(e) {
        alert(`
            Сейчас будет выполнен переход по ссылке.\n
            тег: ${e.target.tagName}\n
```

```
            href: ${e.target.href}\n
        `)
    }
    /*
        // так работать не будет
        const event = new Event("click");
    */
    // создаем событие
    const myevent = new MouseEvent("click");
    // вызов события
    link.dispatchEvent(myevent);
</script>
```

Пользовательские события

Для генерации событий совершенно новых типов (пользовательских событий) следует использовать конструктор **new CustomEvent**.

```
let event = new CustomEvent(type [, options])
```

В демонстрационном примере **example_4** создадим пользовательское событие на узле **document**.

example_4.

```
<script>
    // повесим на document обработчик пользовательского события
    document.addEventListener("myevent", function() {
        alert(this); // document
```

```
    });
    // создадим событие
    let myevent = new CustomEvent("myevent");
    // вызов события
    document.dispatchEvent(myevent);
</script>
```

Технически **CustomEvent** абсолютно идентичен **Event** за исключением одной небольшой детали. У второго аргумента-объекта есть дополнительное свойство **detail**, в котором можно указывать **информацию для передачи в событие**.

С учетом возможных опций синтаксис объявления пользовательского события может быть следующим:

```
let event = new CustomEvent(type, {
    detail: {},
    bubbles: true,
    cancelable: true,
    composed: false
});
```

example_5.

```
<!-- элемент скрыт -->
<div id="myDiv" hidden></div>
<script>
    // повесим на скрытый div обработчик пользовательского события
    // дополнительная информация приходит в обработчик вместе с событием
    myDiv.addEventListener("myevent", function(e) {
```

```
        alert(this); // div
        // дополнительная информация пользовательского события
        alert(e.detail.msg);
        // объект пользовательского события
        console.log(e);
    });
    // создадим объект пользовательского события
    // объект содержит дополнительное свойство detail
    let myevent = new CustomEvent("myevent",
    {detail:{msg:"Инициируем событие на элементе div" }});
    // вызов события
    myDiv.dispatchEvent(myevent);
</script>
```

Свойство **detail** может содержать любые данные. Надо сказать, что никто не мешает и в обычное **new Event** записать любые свойства. Но **CustomEvent** предоставляет специальное поле **detail** во избежание конфликтов с другими свойствами события.

Кроме того, класс события описывает, что это за событие, и если оно не браузерное, а пользовательское, то лучше использовать **CustomEvent**, чтобы явно об этом сказать.

example_6.

```
<p id="elem">Не говори своему другу того, что не должен знать твой враг.</P>
<!--
    для теста можно испоьзовать элемент div
    <div id="elem">Если хочешь подчинить себе все, подчини себя разуму.</div>
```

```
-->
<script>
    // elem = document.getElementById("elem");
    // дополнительная информация приходит в обработчик вместе с событием
    elem.addEventListener("sayhello", function(e) {
        // имя тега, на котором произошло событие
        alert(this.tagName);
        // свойства пользовательского события
        console.log(e.detail.msg);
        console.log(e.detail.text);
    });
    // объект события инициализируем в методе dispatchEvent
    elem.dispatchEvent(new CustomEvent("sayhello", {
        detail: {
            msg: "Инициируем событие на HTML-элементе",
            text: elem.innerText
        }
    }));
</script>
```

Особый интерес вызывает случай, когда необходимо подписать на пользовательское событие несколько HTML-элементов.



Пользовательские события для коллекций

Поработаем с **коллекцией элементов.**

example_7.

```
<div id="citate">
    <p class="p1">Я не расстроен, что ты мне солгал, я расстроен, что отныне не могу тебе верить.</p>
    <p class="p2">Чем выше мы парим, тем меньше кажемся тем, кто не умеет летать</p>
    <!-- ... -->
</div>
<script>
    // получаем коллекцию элементов P
    pars = document.querySelector("#citate").children;
    // на каждый элемент коллекции навесим пользовательское событие
    for (par of pars) {
        /*
            дополнительная информация заключена:
            -- в самом событии
            -- в свойствах текущего элемента -this
        */
        par.addEventListener("userevent", function(e) {
            // текущий элемент коллекции pars
            alert(this + "; class=" + this.className);
```

```
            // свойства пользовательского события
            console.log (e.detail._this);
            console.log (e.detail.msg);
        });
    }
    // создадим объект пользовательского события
    userevent = new CustomEvent("userevent", {
        detail: {
            msg: "Тестируем пользовательское событие",
            _this: this // всегда window
        }
    })
    // переберем элементы коллекции
    for (let par of pars) {
        // вызываем событие на клике по элементу
        par.addEventListener("click", function() {
            par.dispatchEvent(userevent)
        });
    }
</script>
```

example_8.

```
<div id="citate">
    <p class="p1">Я не расстроен, что ты мне солгал, я расстроен,
    что отныне не могу тебе верить.</p>
    <p class="p2">Чем выше мы парим, тем меньше кажемся тем, кто
```

```
    не умеет летать</p>
    <!-- ... -->
</div>
<script>
    // получаем коллекцию элементов P
    pars = document.querySelector("#citate").children;
    // на каждый элемент коллекции навесим пользовательское
    событие
    for (par of pars) {
        /*
            дополнительная информация заключена:
            -- в самом событии
            -- в свойствах текущего элемента -this
        */
        par.addEventListener("userEvent", function(e) {
            // текущий элемент коллекции pars
            alert(this + "; class=" + this.className);
            // свойства пользовательского события
            console.log (e.detail._this);
            console.log (e.detail.msg);
            console.log (e.detail.text);
        });
    }
    // переберем элементы коллекции
    for (let par of pars) {
        // вызываем событие на клике по элементу
```

```
        par.addEventListener("click", function() {
            // пользовательское событие создадим в методе
            dispatchEvent
            par.dispatchEvent(new CustomEvent("userEvent", {
                detail: {
                    msg: "Тестируем пользовательское событие",
                    _this: this, // теперь this текущий элемент
                    text: par.innerText
                }
            }));
        });
    }
</script>
```

Обратите внимание, объект пользовательского события создаем отдельно, но привязываем **this** к каждому элементу.

example_9.

```
<div id="citate">
    <p class="p1">Я не расстроен, что ты мне солгал, я расстроен,
    что отныне не могу тебе верить.</p>
    <p class="p2">Чем выше мы парим, тем меньше кажемся тем, кто
    не умеет летать</p>
    <!-- ... -->
</div>
<script>
    // получаем коллекцию элементов P
    pars = document.querySelector("#citate").children;
```

```
// на каждый элемент коллекции навесим пользовательское
событие
for (par of pars) {
    /*
        дополнительная информация заключена:
        -- в самом событии
        -- в свойствах текущего элемента -this
    */
    par.addEventListener("userevent", function(e) {
    // текущий элемент коллекции pars
    alert(this + "; class=" + this.className);
    // свойства пользовательского события
    console.log (e.detail._this.innerText);
    console.log (e.detail);
    });
}
// создадим объект пользовательского события
userevent = new CustomEvent("userevent", {
    detail: {
        msg: "Тестируем пользовательское событие",
        _this: null
    }
})
// переберем элементы коллекции
for (let par of pars) {
    // вызываем событие на клике по элементу
```

```
        par.addEventListener("click", function() {
            // изменим свойство пользовательского события
            userevent.detail._this = par;
            par.dispatchEvent(userevent)
        });
    }
</script>
```

Политика безопасности браузера

До сих пор наши кнопочки хорошо кликались, а текстовые поля самозаполнялись. Но так бывает не всегда.

Следующим примером еще раз запустим событие **click** по кнопке (элемент button)...

example_10.

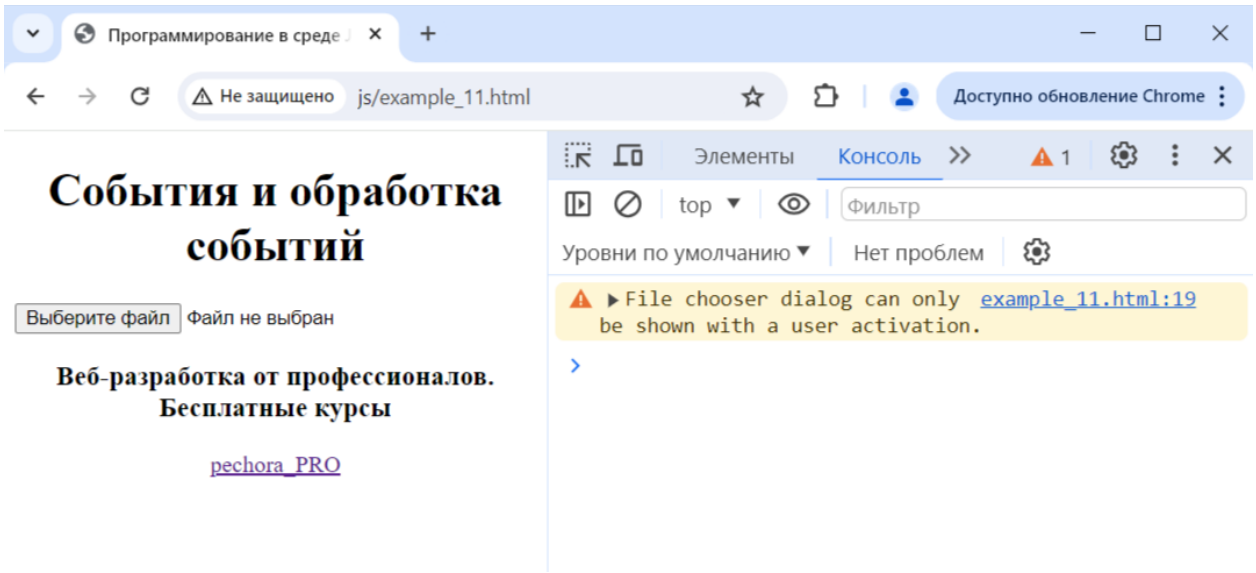
```
<button id="btn" onclick="alert('Упс! ;)')">Сделай Упс...</button><p>
<script>
    // --1
    var clickevent = new MouseEvent("click");
    document.getElementById('btn').dispatchEvent(clickevent);
    // --2
    // document.getElementById('btn').click();
</script>
```

... и сразу попробуем запустить событие **click** по кнопке выбора файла (элемент input type='file').

example_11.

```
<form id="frm">
    <input id="fileInp" type="file">
</form>
<script>
    // --1
    var clickevent = new MouseEvent("click");
    document.getElementById('fileInp').dispatchEvent(clickevent);
    // --2
    // document.getElementById('fileInp').click();
</script>
```

Мой браузер **Google Chrome** данный эксперимент запретил.



Сообщение браузера гласит, что – ...

Важно. Диалоговое окно выбора файла может быть открыто только при **активации пользователем**.

Браузер **Mozilla Firefox** был более дружелюбен. Ну или так – был менее благоразумен.

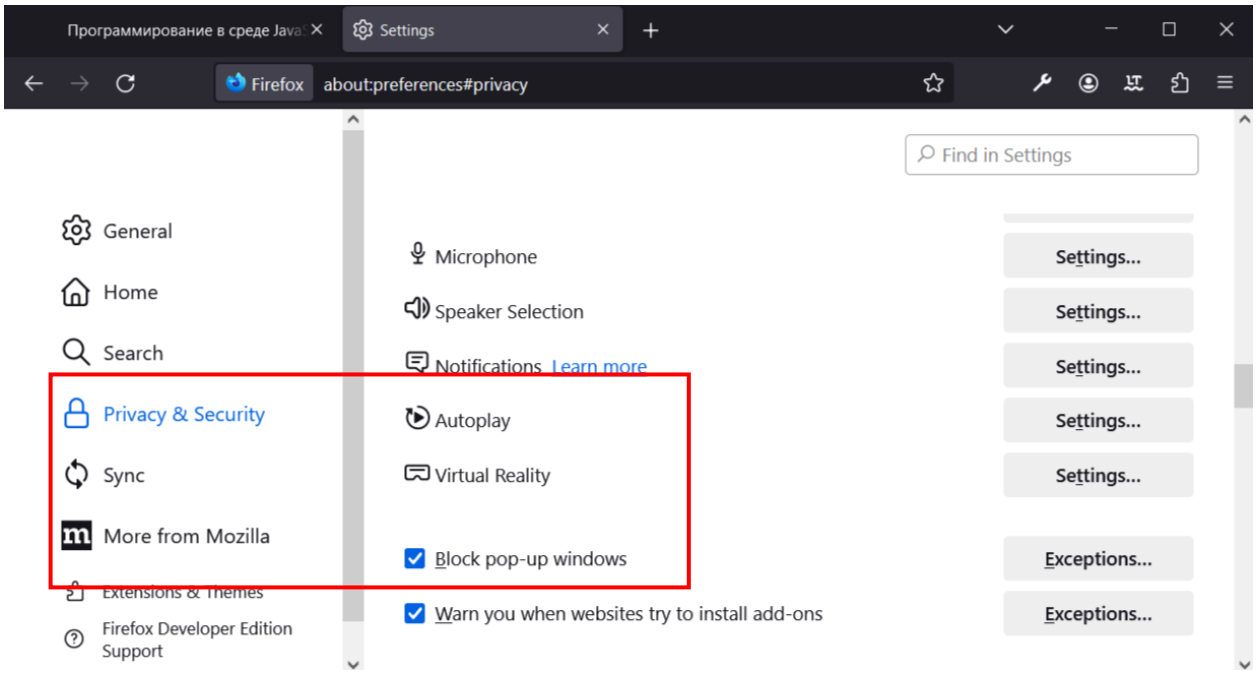
Все зависит от **настроек**. **Но**, будьте готовы к тому, что такое действие будет воспринято браузером подозрительно, и скорее всего запрещено.

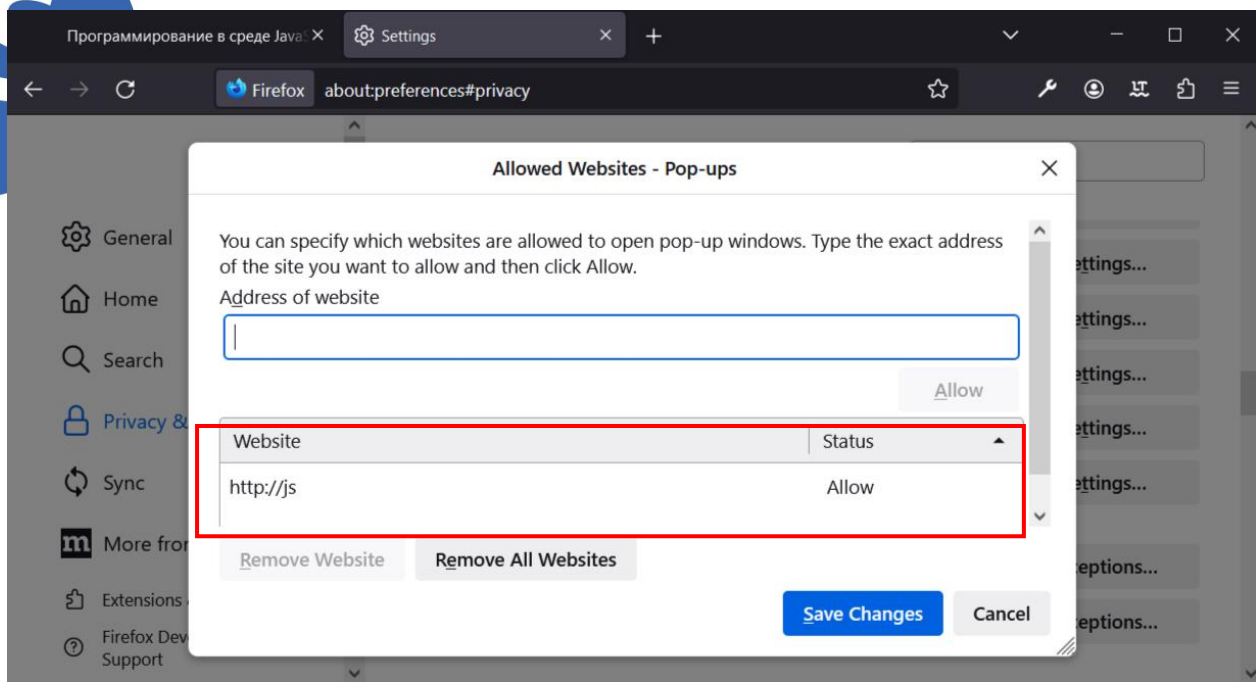
Хотите вразумить **Mozilla Firefox** выполните следующие настройки:

Settings – Privacy & Security – **Block pop-up windows**

После этого диалоговое окно выбора файлов будет блокировано.

> ***Важно***. В моем примере окно диалога все таки открывается в силу указанного исключения: **Exceptions**... – **Allowed Websites – Pop-ups** .





Вы не заставите пользователя вашего сайта открыть настройки браузера и выполнить указанные вами инструкции (ну, а если убеждение ваш конек, не могли бы вы попросить пользователя перевести мне на карту тысяч хотя бы 20-30....?).

Поэтому выполним некоторый обходной маневр. В демонстрационном примере **example_12** рассмотрим какой.

example_12.

```
<form id="frm">
    <input id="fileInp" type="file" hidden>
</form>
<a href="#" id="choise">Выбрать файл</a>
<script>
    // создадим событие клика по ссылке
    document.getElementById('choise').addEventListener("click",
    () => {
        // создадим событие
        var clickevent = new MouseEvent("click");
```

```
        // вызов события открытия диалогового окна
        document.getElementById('fileInp').dispatchEvent(clickeve
        nt);
    });
</script>
```

Может возникнуть логичный вопрос, а зачем городить событийность, если есть стандартный атрибут **for**...

example_13.

```
<label for="fileInp">Выбрать файл</label>
<form id="frm">
    <input id="fileInp" type="file" hidden>
</form>
```

> ***Примечание***. Не стоит путать возможности **браузера** и вас как программиста.

Творческие эксперименты

Для закрепления материала предлагаю несколько примеров, показывающих взаимодействие объекта пользовательского события с элементами DOM браузера.

> ***Примечание***. Следующие демонстрационные примеры желательны, но при отсутствии большого желания экспериментировать предлагаемый код можно пропустить :(и никогда не узнать, как написать метод пользовательского события)



Внимательно выполните предлагаемые примеры, рассмотрите контексты и привязки элементов, читайте комментарии к программному коду.

example_14.

```
<script>
    // глобальные переменные
    var login = "GlobalLogin";
    var pwd = "GlobalPwd";
    var email = "global-email@mail.ru";
    // назначаем элементу document пользовательское событие и обработчик
    document.addEventListener("getdata", function(e) {
        // вывод в браузер
        document.write (
        `
            Контекст <b>${this}</b><br>
            <b>Свойства контекста:</b><br>
            Логин: ${this.login}<br>
            Пароль: ${this.pwd}<br>
            e-mail: ${this.email}<br>
            ---------- <p>
            <b>Свойства пользовательского события:</b><br>
            Логин: ${e.detail.login}<br>
            Пароль: ${e.detail.pwd}<br>
            e-mail: ${e.detail.email}<br>
        `
        );
```

```
    });
    // инициируем пользовательское событие на элементе document
    document.dispatchEvent(new CustomEvent("getdata", {
        detail: {
            // свойства события берем из глобального контекста
            login: this.login,
            pwd: this.pwd,
            email: this.email
        }
    }));
</script>
```

example_15.

```
<h1 id="elemH" data-login="DatasetLogin" data-pwd="DatasetPwd"
data-email="dataset-email@mail.ru">Привет всем!</h1>
<script>
    // назначаем элементу h1#elemH пользовательское событие и
    обработчик
    elemH.addEventListener("getdata", function(e) {
        // вывод в браузер
        document.write (
        `
            Контекст <b>${this}</b><br>
            <b>Свойства элемента H1:</b><br>
            Логин: ${this.dataset.login}<br>
            Пароль: ${this.dataset.pwd}<br>
```

```
            e-mail: ${this.dataset.email}<br>
            ---------- <p>
            <b>Свойства пользовательского события:</b><br>
            Логин: ${e.detail.login}<br>
            Пароль: ${e.detail.pwd}<br>
            e-mail: ${e.detail.email}<br>
        `
        );
    });
    // инициируем пользовательское событие на элементе h1#elemH
    elemH.dispatchEvent(new CustomEvent("getdata", {
        detail: {
            // пытаемся взять свойства из контекста h1#elemH,
            // но так работать не будет !
            login: this.dataset.login,
            pwd:  this.elemH.dataset.pwd,
            email:  this.dataset.email
        }
    }));
</script>
```

example_16.

```
<h1 id="elemH" data-login="DatasetLogin" data-pwd="DatasetPwd"
data-email="dataset-email@mail.ru">Привет всем!</h1>
<form id="frm">
<input type="text" name="login" value="FormLogin"><p>
```

```
<input type="password" name="pwd" value="FormPwd"><p>
<input type="text" name="email" value="form-pwd@mail.ru"><p>
<button>Отправить данные</button>
</form>
<script>
    // назначаем элементу document пользовательское событие и
    обработчик
    document.addEventListener("getdata", function(e) {
        // вывод в браузер
        document.write (
        `
            Контекст <b>${this}</b><br>
            <b>Свойства элемента H1:</b><br>
            Логин: ${elemH.dataset.login}<br>
            Пароль: ${elemH.dataset.pwd}<br>
            e-mail: ${elemH.dataset.email}<br>
            ---------- <p>
            <b>Свойства пользовательского события:</b><br>
            Логин: ${e.detail.login}<br>
            Пароль: ${e.detail.pwd}<br>
            e-mail: ${e.detail.email}<br>
        `
        );
    });
    // инициируем пользовательское событие на элементе document
    document.dispatchEvent(new CustomEvent("getdata", {
```

```
        detail: {
            // свойства события берем из элементов формы
            login: frm.elements['login'].value,
            pwd: frm.elements['pwd'].value,
            email: frm.elements['email'].value
        }
    }));
</script>
```

example_17.

```
<h1 id="elemH" data-login="DatasetLogin" data-pwd="DatasetPwd"
data-email="dataset-email@mail.ru">Привет всем!</h1>
<form id="frm">
    <input type="text" name="name" value="Иван"><p>
    <input type="text" name="surname" value="Бездомный"><p>
    <input type="text" name="login" value="FormLogin"><p>
    <input type="password" name="pwd" value="FormPwd"><p>
    <input type="text" name="email" value="form-pwd@mail.ru"><p>
    <button>Отправить данные</button>
</form>
<script>
    // назначаем элементу document пользовательское событие и
    обработчик
    document.addEventListener("getdata", function(e) {
        // вывод в браузер
        document.write (
```

```
        `
            Контекст <b>${this}</b><br>
            <b>Свойства элемента H1:</b><br>
            Логин: ${elemH.dataset.login}<br>
            Пароль: ${elemH.dataset.pwd}<br>
            e-mail: ${elemH.dataset.email}<br>
            ---------- <p>
            <b>Свойства пользовательского события:</b><br>
            Логин: ${e.detail.login}<br>
            Пароль: ${e.detail.pwd}<br>
            e-mail: ${e.detail.email}<br>
        `
    );
    document.write (e.detail.sayHello())
});
// инициируем пользовательское событие на элементе document
document.dispatchEvent(new CustomEvent("getdata", {
    detail: {
        // свойства события берем из элементов формы
        name: frm.elements['name'].value,
        surname: frm.elements['surname'].value,
        pwd: frm.elements['pwd'].value,
        login: frm.elements['login'].value,
        email: frm.elements['email'].value,
        // создаем метод пользовательского события
        sayHello: function(){
```

```
                name = this.name;
                surname = this.surname;
                return `<h2>Уважаемый ${name} ${surname},
                здравствуйте! Так что же такое шизофрения?</h2>`;
            }
        }
    }));
</script>
```

В **самостоятельных работах** к уроку выполним следующие задания:

- Создание сценария для вызова окна открытия файлов при клике по пиктограмме изображения.
- Создание сценария, запрашивающего разрешение на переход при клике по ссылке.
- Создание пользовательского события, создающего массив объектов из таблицы при загрузке документа.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.



К **курсу**:

- **Дополнительный практикум.**

